// PART 3 OF 4

GOLD BATCH

nb_gold_build_sql · 12 Delta Tables · 17 Cells · 0 Lines PySpark · Star Schema + KPI Marts

```
/* Star schema:
/* Facts (5) : gold_fact_orders, gold_fact_inventory, gold_fact_shipments, gold_fact_labor, gold_fact_dock
/* Dims (4) : gold_dim_customer, gold_dim_product, gold_dim_warehouse, gold_dim_date
/* KPI (3) : gold_kpi_supplier, gold_kpi_returns, gold_kpi_iot_summary
/*
/* Build order: Dimensions (1-4) → Facts (5-9) → KPI Marts (10-12) → OPTIMIZE (13-14) → VACUUM (15)
*/
```

// nb_gold_build_sql

■■ DIMENSIONS ■■

Build dims first — facts join to these

// nb_gold_build_sql

CELL 1 — gold.gold_dim_date

source: silver.silver_date_dim · anchor for all Power BI time-intelligence

```
CREATE OR REPLACE TABLE gold.gold_dim_date
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
date_key, full_date, day_of_week, day_name, week_num,
month_num, month_name, month_label, quarter, year,
fiscal_period, is_weekend, is_holiday,
current_timestamp() AS _loaded_at
FROM silver.silver_date_dim;

SELECT 'gold.gold_dim_date' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_date;
```

// nb_gold_build_sql

CELL 2 — gold.gold_dim_customer

source: silver.silver_customers · derived: full_name

```
CREATE OR REPLACE TABLE gold.gold_dim_customer
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
customer_id, first_name, last_name,
CONCAT(first_name, ' ', last_name) AS full_name,
segment, account_status, credit_limit,
city, state, region, country, customer_since_year, created_date,
current_timestamp() AS _loaded_at
FROM silver.silver_customers;

SELECT 'gold.gold_dim_customer' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_customer;
```

// nb_gold_build_sql

CELL 3 — gold.gold_dim_product

source: silver.silver_products · derived: price_tier (Economy / Mid / Premium)

```sql
CREATE OR REPLACE TABLE gold.gold_dim_product
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
sku, product_name, category, subcategory, supplier_id,
unit_cost, unit_price, margin_pct, weight_kg, volume_cuft,
reorder_point, reorder_qty, lead_time_days, is_hazmat, is_perishable,
CASE
 WHEN unit_price >= 500 THEN 'Premium'
 WHEN unit_price >= 100 THEN 'Mid'
 ELSE 'Economy'
 END AS price_tier,
 current_timestamp() AS _loaded_at
 FROM silver.silver_products;

 SELECT 'gold.gold_dim_product' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_product;
```

// nb_gold_build_sql

CELL 4 — gold.gold_dim_warehouse

source: bronze.raw_warehouses · small reference table — reads Bronze directly

```sql
CREATE OR REPLACE TABLE gold.gold_dim_warehouse
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
SELECT *, ROW_NUMBER() OVER (PARTITION BY warehouse_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_warehouses WHERE warehouse_id IS NOT NULL
)
SELECT
warehouse_id,
name AS warehouse_name,
region, city, state, country, manager_id,
CAST(capacity_sqft AS DOUBLE) AS capacity_sqft,
CAST(num_docks AS INT) AS num_docks,
CAST(num_zones AS INT) AS num_zones,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

SELECT 'gold.gold_dim_warehouse' AS tbl, COUNT(*) AS rows FROM gold.gold_dim_warehouse;
```

// nb_gold_build_sql

■■ FACT TABLES ■■

Dimensions must exist before running these cells

// nb_gold_build_sql

CELL 5 — gold.gold_fact_orders

grain: 1 row per order_id · sources: silver_orders + silver_order_lines (agg) + silver_shipments (agg)

```sql
CREATE OR REPLACE TABLE gold.gold_fact_orders
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH line_agg AS (
-- Roll 128,899 line rows up to order grain
SELECT
order_id,
SUM(line_value) AS total_line_value,
SUM(line_margin) AS total_margin,
SUM(qty_ordered) AS total_qty_ordered,
SUM(qty_fulfilled) AS total_qty_fulfilled,
SUM(qty_backordered) AS total_qty_backordered,
AVG(fulfillment_rate) AS avg_fulfillment_rate,
COUNT(DISTINCT sku) AS distinct_skus
FROM silver.silver_order_lines
GROUP BY order_id
),
ship_agg AS (
-- Roll multiple shipments per order to order grain
SELECT
order_id,
MIN(ship_date) AS first_ship_date,
SUM(shipping_cost) AS total_shipping_cost,
MAX(CAST(is_on_time AS INT)) = 1 AS was_on_time,
SUM(delay_days) AS total_delay_days,
COUNT(shipment_id) AS num_shipments
FROM silver.silver_shipments
GROUP BY order_id
)
SELECT
o.order_id,
o.customer_id,
o.warehouse_id,
CAST(DATE_FORMAT(o.order_date,'yyyyMMdd') AS INT) AS date_key,
o.order_date, o.order_month, o.order_year,
o.status, o.priority, o.order_value,
l.total_line_value,
l.total_margin,
CASE WHEN l.total_line_value > 0
THEN l.total_margin / l.total_line_value
ELSE 0.0 END AS margin_pct,
l.total_qty_ordered, l.total_qty_fulfilled, l.total_qty_backordered,
l.avg_fulfillment_rate, l.distinct_skus,
s.first_ship_date, s.total_shipping_cost, s.was_on_time,
s.total_delay_days, s.num_shipments,
DATEDIFF(s.first_ship_date, o.order_date) AS days_to_ship,
o.days_to_required,
current_timestamp() AS _loaded_at
```

```
FROM silver.silver_orders o
LEFT JOIN line_agg l ON o.order_id = l.order_id
LEFT JOIN ship_agg s ON o.order_id = s.order_id;

SELECT 'gold.gold_fact_orders' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_orders;
```

```
// nb_gold_build_sql
```

CELL 6 — gold.gold_fact_inventory

grain: 1 row per inventory_id · derived: stock_status, days_since_counted

```
CREATE OR REPLACE TABLE gold.gold_fact_inventory
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
inventory_id, sku, warehouse_id, location_id,
qty_on_hand, qty_reserved, qty_available, qty_in_transit,
unit_cost, inventory_value, below_reorder, reorder_point, utilization_rate,
last_counted_at, last_received_at, category, subcategory, product_name,
DATEDIFF(CURRENT_DATE(), TO_DATE(last_counted_at)) AS days_since_counted,
CASE
WHEN qty_available = 0 THEN 'Out of Stock'
WHEN below_reorder = TRUE THEN 'Low Stock'
ELSE 'In Stock'
END AS stock_status,
current_timestamp() AS _loaded_at
FROM silver.silver_inventory;

SELECT 'gold.gold_fact_inventory' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_inventory;
```

```
// nb_gold_build_sql
```

CELL 7 — gold.gold_fact_shipments

grain: 1 row per shipment_id · derived: date_key, ship_month

```
CREATE OR REPLACE TABLE gold.gold_fact_shipments
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
shipment_id, order_id, carrier_id, carrier_name, service_level,
origin_warehouse_id, ship_date, estimated_delivery, actual_delivery, status,
weight_lbs, shipping_cost, is_on_time,
COALESCE(delay_days, 0) AS delay_days,
num_packages, cost_per_lb, avg_transit_days,
CAST(DATE_FORMAT(ship_date,'yyyyMMdd') AS INT) AS date_key,
DATE_FORMAT(ship_date,'yyyy-MM') AS ship_month,
current_timestamp() AS _loaded_at
FROM silver.silver_shipments;

SELECT 'gold.gold_fact_shipments' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_shipments;
```

```
// nb_gold_build_sql
```

CELL 8 — gold.gold_fact_labor

grain: 1 row per shift_id · derived: date_key, cost_per_pick

```
CREATE OR REPLACE TABLE gold.gold_fact_labor
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
```

```
SELECT
shift_id, employee_id, warehouse_id, zone_assigned, shift_type,
shift_date, shift_month, hours_worked, picks_completed, units_processed,
picks_per_hour, units_per_hour, labor_cost,
role, first_name, last_name, hourly_rate,
CAST(DATE_FORMAT(shift_date,'yyyyMMdd') AS INT) AS date_key,
CASE WHEN picks_completed > 0
THEN labor_cost / picks_completed
ELSE NULL END AS cost_per_pick,
current_timestamp() AS _loaded_at
FROM silver.silver_labor_shifts;

SELECT 'gold.gold_fact_labor' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_labor;
```

```
// nb_gold_build_sql
```

CELL 9 — gold.gold_fact_dock

grain: 1 row per dock_id · derived: date_key

```
CREATE OR REPLACE TABLE gold.gold_fact_dock
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
dock_id, warehouse_id, carrier_id, shipment_id,
activity_type, activity_date, activity_hour,
duration_minutes, num_pallets, pallets_per_hour, is_inbound,
CAST(DATE_FORMAT(activity_date,'yyyyMMdd') AS INT) AS date_key,
current_timestamp() AS _loaded_at
FROM silver.silver_dock_activity;

SELECT 'gold.gold_fact_dock' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_dock;
```

// nb_gold_build_sql

■■ KPI MARTS ■■

// nb_gold_build_sql

CELL 10 — gold.gold_kpi_supplier

grain: supplier × month · purpose: Supplier Scorecard report

```sql
CREATE OR REPLACE TABLE gold.gold_kpi_supplier
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
sp.perf_id, sp.supplier_id, sp.supplier_name,
CAST(s.is_preferred AS BOOLEAN) AS is_preferred,
sp.country, sp.period_month, sp.period_year, sp.month_label,
sp.total_pos, sp.on_time_deliveries, sp.on_time_rate,
sp.fill_rate, sp.defect_rate, sp.avg_lead_days,
sp.composite_score, sp.score_tier,
s.reliability_score,
current_timestamp() AS _loaded_at
FROM silver.silver_supplier_performance sp
LEFT JOIN bronze.raw_suppliers s ON sp.supplier_id = s.supplier_id;

SELECT 'gold.gold_kpi_supplier' AS tbl, COUNT(*) AS rows FROM gold.gold_kpi_supplier;
```

// nb_gold_build_sql

CELL 11 — gold.gold_kpi_returns

grain: 1 row per return_id · derived: loss_amount (non-resellable only)

```sql
CREATE OR REPLACE TABLE gold.gold_kpi_returns
USING DELTA
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT
r.return_id, r.order_id, r.sku,
p.product_name, p.category,
r.qty_returned, r.refund_amount, r.return_date,
DATE_FORMAT(r.return_date,'yyyy-MM') AS return_month,
YEAR(r.return_date) AS return_year,
r.reason_code, r.reason_group,
r.condition, r.disposition, r.is_resellable, r.days_to_restock,
p.unit_cost,
-- non-resellable items written off at cost
CASE WHEN r.is_resellable = FALSE
THEN r.qty_returned * p.unit_cost
ELSE 0.0 END AS loss_amount,
current_timestamp() AS _loaded_at
FROM silver.silver_returns r
LEFT JOIN silver.silver_products p ON r.sku = p.sku;

SELECT 'gold.gold_kpi_returns' AS tbl, COUNT(*) AS rows FROM gold.gold_kpi_returns;
```

// nb_gold_build_sql

CELL 12 — gold.gold_kpi_iot_summary (BATCH)

grain: device_id × warehouse_id × zone × event_date · hourly streaming version → Part 4

```sql
CREATE OR REPLACE TABLE gold.gold_kpi_iot_summary
USING DELTA
```

```
TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH cast_events AS (
SELECT
event_id, device_id, device_type, warehouse_id, zone, event_type,
TO_DATE(event_timestamp) AS event_date,
CAST(speed_mph AS DOUBLE) AS speed_mph,
CAST(battery_pct AS DOUBLE) AS battery_pct,
CAST(temperature_f AS DOUBLE) AS temperature_f,
CAST(carrying_qty AS INT) AS carrying_qty
FROM bronze.raw_iot_events
WHERE event_id IS NOT NULL
)
SELECT
device_id, device_type, warehouse_id, zone, event_date,
CAST(DATE_FORMAT(event_date,'yyyyMMdd') AS INT) AS date_key,
COUNT(event_id) AS total_events,
ROUND(AVG(speed_mph), 2) AS avg_speed_mph,
ROUND(MAX(speed_mph), 2) AS max_speed_mph,
ROUND(AVG(battery_pct), 2) AS avg_battery_pct,
ROUND(MIN(battery_pct), 2) AS min_battery_pct,
ROUND(AVG(temperature_f), 2) AS avg_temp_f,
SUM(carrying_qty) AS total_carrying_qty,
SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END) AS alert_count,
SUM(CASE WHEN event_type = 'IDLE' THEN 1 ELSE 0 END) AS idle_count,
SUM(CASE WHEN event_type = 'MOVE' THEN 1 ELSE 0 END) AS move_count,
ROUND(
SUM(CASE WHEN event_type = 'ALERT' THEN 1 ELSE 0 END)
/ CAST(COUNT(event_id) AS DOUBLE), 4
) AS alert_rate,
current_timestamp() AS _loaded_at
FROM cast_events
GROUP BY device_id, device_type, warehouse_id, zone, event_date;

SELECT 'gold.gold_kpi_iot_summary' AS tbl, COUNT(*) AS rows FROM gold.gold_kpi_iot_summary;
```

// nb_gold_build_sql

■■ POST-BUILD: OPTIMIZE + ZORDER + VACUUM ■■

Run after all 12 Gold tables are written · stop stream first if running

// nb_gold_build_sql

CELL 13 — OPTIMIZE + ZORDER: Fact Tables

```
OPTIMIZE gold.gold_fact_orders ZORDER BY (order_date, customer_id);
OPTIMIZE gold.gold_fact_inventory ZORDER BY (sku, warehouse_id);
OPTIMIZE gold.gold_fact_shipments ZORDER BY (ship_date, carrier_id);
OPTIMIZE gold.gold_fact_labor ZORDER BY (shift_date, warehouse_id);
OPTIMIZE gold.gold_fact_dock ZORDER BY (activity_date, warehouse_id);
```

// nb_gold_build_sql

CELL 14 — OPTIMIZE + ZORDER: KPI Marts

```
OPTIMIZE gold.gold_kpi_supplier ZORDER BY (period_year, period_month);
OPTIMIZE gold.gold_kpi_returns ZORDER BY (return_date, reason_code);
OPTIMIZE gold.gold_kpi_iot_summary ZORDER BY (event_date, warehouse_id);
```

// nb_gold_build_sql

CELL 15 — VACUUM: All Gold Tables (retain 7 days / 168 hours)

```
VACUUM gold.gold_fact_orders RETAIN 168 HOURS;
VACUUM gold.gold_fact_inventory RETAIN 168 HOURS;
VACUUM gold.gold_fact_shipments RETAIN 168 HOURS;
VACUUM gold.gold_fact_labor RETAIN 168 HOURS;
VACUUM gold.gold_fact_dock RETAIN 168 HOURS;
VACUUM gold.gold_dim_customer RETAIN 168 HOURS;
VACUUM gold.gold_dim_product RETAIN 168 HOURS;
VACUUM gold.gold_dim_warehouse RETAIN 168 HOURS;
VACUUM gold.gold_dim_date RETAIN 168 HOURS;
VACUUM gold.gold_kpi_supplier RETAIN 168 HOURS;
VACUUM gold.gold_kpi_returns RETAIN 168 HOURS;
VACUUM gold.gold_kpi_iot_summary RETAIN 168 HOURS;
```

// nb_gold_build_sql

CELL 16 — Full Gold Validation

all 12 tables must show ✓ OK

```
SELECT tbl, rows,
CASE WHEN rows > 0 THEN '✓ OK' ELSE '✗ EMPTY' END AS status
FROM (
SELECT 'gold_fact_orders' AS tbl, COUNT(*) AS rows FROM gold.gold_fact_orders UNION ALL
SELECT 'gold_fact_inventory', COUNT(*) FROM gold.gold_fact_inventory UNION ALL
SELECT 'gold_fact_shipments', COUNT(*) FROM gold.gold_fact_shipments UNION ALL
SELECT 'gold_fact_labor', COUNT(*) FROM gold.gold_fact_labor UNION ALL
SELECT 'gold_fact_dock', COUNT(*) FROM gold.gold_fact_dock UNION ALL
SELECT 'gold_dim_customer', COUNT(*) FROM gold.gold_dim_customer UNION ALL
SELECT 'gold_dim_product', COUNT(*) FROM gold.gold_dim_product UNION ALL
SELECT 'gold_dim_warehouse', COUNT(*) FROM gold.gold_dim_warehouse UNION ALL
SELECT 'gold_dim_date', COUNT(*) FROM gold.gold_dim_date UNION ALL
SELECT 'gold_kpi_supplier', COUNT(*) FROM gold.gold_kpi_supplier UNION ALL
SELECT 'gold_kpi_returns', COUNT(*) FROM gold.gold_kpi_returns UNION ALL
SELECT 'gold_kpi_iot_summary', COUNT(*) FROM gold.gold_kpi_iot_summary
) t
ORDER BY tbl;
```

// nb_gold_build_sql

CELL 17 — Quick Sanity: Revenue + OTD Rate

```sql
SELECT
ROUND(SUM(order_value), 2) AS total_revenue,
ROUND(AVG(margin_pct) * 100, 2) AS avg_margin_pct,
COUNT(*) AS total_orders,
SUM(CASE WHEN was_on_time THEN 1 ELSE 0 END) AS orders_on_time,
ROUND(
SUM(CASE WHEN was_on_time THEN 1 ELSE 0 END)
/ CAST(COUNT(*) AS DOUBLE) * 100, 2
) AS otd_rate_pct
FROM gold.gold_fact_orders;
```